

WorkLoom（织元）· 企业 Agent IM 通用底座技术方案

以 DeepSeek Harness 为运行时地基的「企业 Agent IM——行业可装配 Agent 发行版」技术详细设计

文档版本	V3.0（企业 Agent IM 视角重整版 · 纯净终稿）
日期	2026-08-15
唯一事实源	《WorkLoom 织元 · Agent 时代的 IM 产品需求文档（PRD）V2.1（企业 Agent IM 全篇贯通版）》；行业 PRD：《易飞轮 CodexWork PRD V1.1》《未来前台 CodexWork PRD V2.0》
上游依据	《WorkLoom 织元通用底座技术方案 V2.0》（2026-08-14，本版直接前身，技术内容全量继承）·《DeepSeek Harness 深度调研报告 V1.0》·《WorkLoom × DeepSeek Harness 深度融合方案（路径 A）V1.0》
交付目标	可直接交给 AI Coding 开发的可编码级技术详细设计
V3.0 变更	视角与主体切换：全文从「企业 Agent IM」立场重新组织——事件架构更名「消息架构」、围栏更名「行动权限」、审批更名「原生消息类型」、夜班更名「Agent 在线时长」；总体架构对齐 PRD V2.1 附录 A 的 IM 五层；需求对齐总表改挂 PRD V2.1 模块编号。 技术选型、数据模型、接口契约、验收门槛零变更 （除第 3/10 章按 2026-08-15 复核结论更新的 LiteLLM 供应链注记与 Langfuse 附注）。

0 文档说明

阅读约定：需求回指标记采用 PRD V2.1 编号体系（F=功能、L=限制、E=异常、US=用户故事、G=全局验收口径、M=底座模块、P=页面）；行业版编号加前缀 H-（酒店版）/ Y-（营销版）。PRD 未覆盖而技术必需的项，集中列于第 13 章「技术新增量」，不混入主线。本文档**不含任何排期与工期**，只给模块依赖拓扑与可复核的验收门槛。

IM 本体对应（与 PRD V2.1 术语表一致）：消息 = 五元事件
（Who×Context×Object×Decision×RuleImpact + receipt + model_trace）；会话 = 任务线程（自

带状态机)；成员 = 人机混编 (人类角色 + Agent preset)；审批卡片 = 原生消息类型；回执 = 外部系统生效确认+证据快照；在线 = 夜班状态机。本方案所有组件均为这套消息协作网络协议的工程实现。

1 产品本质与技术命题

1.1 本质判断：WorkLoom 是什么

WorkLoom（织元）不是工作台（工作台只是它的 IM 客户端皮肤），不是传统 SaaS（系统预设菜单，它按意图动态组装能力），也不是单个智能体（Agent 班组只是这张网络里的「成员」）。它的本质是一张 **企业 Agent IM**——「**行业可装配的 Agent 发行版**」(**Industry-Composable Agent Distribution**)：一张以消息（五元事件）为唯一事实源、以规则围栏为行动权限边界、以三态会话（Ask/Agent/Quest）为人机分工范式的人机协作网络；人与 Agent 在同一 Workspace 中以同一种消息协议协作——人授权与裁决，Agent 执行与留痕。行业差异（技能、工具、围栏阈值、工作台 UI、Agent 班组）全部以 Bundle 形式装配，客户差异以 Patch 收尾。

```
WorkLoom = DeepSeek Harness (dsh 公共运行时地基)
+ WorkLoom Base Bundle (企业 Agent IM 引擎层 = 自有核心资产)
+ 行业 Bundle (营销版 / 酒店版 / N 个未来行业 = 行业通讯录+群规+皮肤)
+ 客户级 Patch (私有化定制)

WorkLoom for Hotels    ≈ dsh --profile workloom-hotel
WorkLoom for Marketing ≈ dsh --profile workloom-marketing
```

商业模式类比：RedHat 之于 Linux——运行时地基开源共荣，发行版与行业 bundle 闭源收费。

1.2 两份行业 PRD 抽象出的通用骨架

抽象层	营销版（FlyData）	酒店版（Workdata）	通用版组件（IM 本体）
数据底座	一客一档 + 五元事件图谱 + 统一记忆	一店一档 + 五元事件图谱 + 统一记忆	flydata-core（消息总线）
人机范式	Ask / Agent / Quest 三态互切	同	三态会话引擎（dsh preset 原生承接）
治理	常规自动 / 敏感必审 / 高危熔断	同	fence-engine（行动权限）

王牌场景	AI 夜班员工 + 08:30 清晨决策包	同	night-shift (Agent 在线时长)
执行体	运营 8 + 私域 5 智能体	调价/巡检/对账等 Agent	角色化 Agent 班组 (行业 preset = 行业通讯录)
资产沉淀	技能广场 + 意识系统	同	方法论资产层 (Skills + 意识插件)
触达	统一审查面板 + IM 卡片审批	同	review-console (原生审批消息)
商业化	版本分级 + 峰谷积分	同	tenancy + model-router

1.3 技术命题

把这张企业 Agent IM 落成可开发系统，技术方案须回答五件事：

#	命题	IM 视角表述
1	运行时复用边界 ：哪些交给 dsh (Loop、工具流水线、会话日志、审批瀑布、多 Agent 原语、多模型适配)，哪些做成自有插件 (护城河)，哪些自建服务 (数据面、前端) —— 路径 A 已定论：约 60% 复用、30% 插件化 (护城河所在)、10% 自建	IM 的「传输层」复用公共地基；「协议层」(消息 Schema、行动权限、审批消息类型) 必须自有
2	消息架构 (原称事件架构)：五元业务事件如何与 dsh 运行时事件共存于同一条 append-only 日志，使「操控留痕 100%」「决策链路可导出」「禁止无痕回滚」从应用层纪律升级为运行时不变量	消息总线的实现：消息即事件，凡入会话者皆已落盘 (G8)
3	行动权限架构 (原称治理架构)：围栏三级管控如何落到 tools/pre-execute 瀑布 + 单调守卫，保证「可收紧不可放宽」且 PTC 子调用无后门	IM 的权限系统从「谁能看」升级为「谁能动手」
4	多租户与版本裁剪 ：渠道总控台数据隔离、版本能力矩阵如何用 bundle 组合 + seam 提供方 + 服务端鉴权三层实现	一张消息网络服务多组织：组织间物理隔离，组织内人机混编
5	行业复制机制 ：行业 Bundle 的标准结构与开发指南，使第三个行业的落地成本趋近于「写 bundle，不改底座」	换行业 = 给这张 IM 换一套行业成员、行业群规与行业皮肤

1.4 关键假设与业务量级

#	假设	取值	影响面
A1	部署形态	云端 SaaS 多租户为主；门店/客户侧可有边缘节点；全 VPC 私有化为一等形态（PRD V2.1 安全专章）	数据层/权限/部署
A2	业务量级	数十客户 / 300+ 账号（营销版）；单店事件写入 <5000 条/日（酒店版）；夜班事件千级每晚；单工作区并发任务 ≤10	不上 K8s、不上消息队列、不拆微服务的依据；单机 API + Worker + Postgres 即可承载
A3	平台通道	有官方 API 的平台 API 优先；无 API 平台走浏览器自动化（Stagehand 剧本为主路径，browser-use 探索兜底）	执行面分层（6.6）
A4	技术栈	运行时与服务层 TypeScript（dsh 同生态）；边缘节点与桌面 GUI 操控允许 Python 进程；前端 TypeScript（React + Tauri 桌面 / Taro 小程序）	全篇
A5	大模型	商业模型为主，经 LiteLLM/dsh 适配层路由；不训练自研模型；素材生成外接第三方模型 API	成本模型
A6	边缘节点硬件	单店边缘节点一台普通办公 PC（8C16G + 512G SSD）即可运行全部本地组件	端-边-云分层

2 总体架构

2.1 产品五层 × 部署四层（对齐 PRD V2.1 附录 A 的企业 Agent IM 五层）

产品视角（企业 Agent IM 五层，PRD V2.1 附录 A）：

IM 客户端层	三态会话派遣中心 · 清晨决策包消息 · 待办收件箱 · IM 审批卡片 (P1-P7 外壳 + 行业页
Agent 成员层	行业 Agent 班组(preset, 通讯录一等成员) · 夜班班组 · 巡检(只读) · 触发器
行动权限层	围栏引擎 (三级/单调基线/版本化/dry-run) · 安全网关 (权限/脱敏/授权)
消息总线层	五元消息图谱(append-only+哈希链) · 一X一档 · 组织记忆(归因/校准/脱敏回流)
资源层	模型路由 (记忆优先/分级/峰谷/降级链/计量) · 租户与版本裁剪 · 积分 · 凭据

实现视角（L0-L4 叠加堆栈）：

L4 客户级 Patch (cordis.patch.yml: 私有围栏阈值 / VPC seam 指向 / 品牌定制)	
L3 行业 Bundle (按 profile 激活, 可多选) = 行业通讯录 + 行业群规 + 行业皮肤	
├─ workloom-marketing: 13+2 智能体 preset、平台适配器、营销围栏包、	
│ 官方技能套件、四工作台 UI	
└─ workloom-hotel: 7 角色 preset、Atlas 浏览器与 PMS 工具、	
酒店围栏包、驾驶舱 UI	
L2 WorkLoom Base Bundle (企业 Agent IM 引擎层 · 自有核心资产 · 闭源)	
flydata-core	消息总线: 五元消息扩展 + 档案服务 + 组织记忆 + 脱敏回流
fence-engine	行动权限: 三级围栏瀑布段 + 围栏版本化 + dry-run 回放校验
night-shift	在线时长: 夜班调度 + 清晨决策包投影 + 一键暂停
review-console	原生审批消息: 统一审查 + IM 审批连接器 + 手势回流校准
model-router	分级路由 / 峰谷调度 / 降级链 / 逐消息计量
tenancy	多租户隔离 + 版本分级裁剪 + 积分计费
L1 DeepSeek Harness v0.1+ (MIT, 锁定版本 + fork 镜像)	
dsh-base: Agent Loop / 工具注册表与流水线 / append-only 会话日志 /	
沙箱与审批策略 / ctx.llm 多模型 seam / ctx.jobs / Skills / MCP	
Cordis 元框架: 插件生命周期 · 可撤销效果 · 响应式协效应 · HMR	
L0 基础设施 (seam 提供方可替换)	
PostgreSQL+pgvector / 对象存储 / E2B·Daytona 沙箱 / Langfuse /	
LiteLLM(VPC 本地模型) / IM 通道 (钉钉·企微·飞书·Slack)	

2.2 控制面 / 数据面分离

VPC 版在技术上等价于一个架构动作：控制面（会话编排、审批队列、版本分级、计量）与数据面（消息库、档案、PII、执行沙箱）分离，全部跨面流量经脱敏网关。落点：数据面组件全部是 seam 提供方。

部署形态	控制面	数据面	模型
SaaS 版	WorkLoom 云	WorkLoom 云（共享存储提供方，租户隔离）	云端多模型
混合版	WorkLoom 云	客户内网	云端+内网混合路由
全 VPC 版	客户内网（同一镜像）	客户内网	本地模型（LiteLLM 代理 vLLM/Ollama）

三形态同一镜像交付，差异仅在 profile 的 bundle 组合与 patch——这正是 dsh Profile/Bundle/Patch 分层机制的原生能力（对齐 PRD 安全专章 S-F1）。

2.3 运行时关键链路（一次 Quest 会话全生命周期）

```
用户目标（文本/语音/截图/快照）
→ 安全网关（权限校验 + PII 脱敏 + 附件扫描） [tenancy / flydata-core]
→ 意图路由（Ask / Agent / Quest 分类，可见可改） [router 插件 → agent/pre-step ]
→ preset 装配（工具集 + 提示词段 + 围栏声明 + 档案上下文） [行业 bundle = 拉起会话成员]
→ Agent Loop (dsh) 逐 step:
    每步工具调用 → tools/pre-execute 瀑布:
        审批策略段 → fence-engine 三级判定(allow/ask/deny)
            → 单调守卫收敛（deny 不可逆）→ 沙箱包裹 → 超时/计量
        → tools/execute → 工具本体（MCP / 原生插件 / PTC 子调用同管道）
        → tools/post-execute（结果改写 / 追加上下文）
        → 五元消息写入 flydata-core（必经安全网关，含执行回执位）
→ ask 动作 → 挂起 → review-console（端内或 IM 审批卡片）→ 手势回写事件
→ 交付：变更报告 + 决策链路（消息日志投影生成，非额外报表）
→ 手势回流校准组织记忆（批准/修改/驳回 = 权重 1/2/3，最高权重信号）
```

3 组件域选型总表（开源复用优先 • 2026-08-15 复核）

核验纪律：凡列入运行时依赖的组件均已核验官方仓库 License 与近期维护状态；归档/停更项目一律不进入运行时；AGPL 组件仅以独立进程形态使用，不链接进分发。全部依赖 pin 版本 + lock 入库；dsh 锁定版本并维护 fork 内部镜像；每季度复查组件活性与上游破坏性变更。

#	组件域	选型	官方仓库 / License
1	Agent 运行时底座	DeepSeek Harness (dsh) v0.1 锁定版 + fork 镜像	官方仓库 + 内部 fork 镜像（锁定 v0.1）• MIT
2	插件元框架	Cordis（随 dsh）	cordiverse/cordis • MIT
3	模型网关（VPC 本地模型/第三方聚合）	LiteLLM Proxy（-stable 镜像， 按哈希 pin，不用浮动 tag；须 ≥1.83.7，当前核验 v1.89.3-stable ） 2026-08-15 复核：2026-03 曾发生 PyPI 供应链投毒（1.82.7/1.82.8）；CVE-2026-42208 SQL 注入（修复于 1.83.7，CISA KEV 在列）——故强制哈希 pin + cosign 验签 + 网络隔离 + 虚拟密钥分域轮换，仅作 VPC 模型代理不暴露公网	github.com/BerriAI/litellm • MIT（核心）
4	组织记忆引擎	mem0 自托管内核 + 自有归因/脱敏层	github.com/mem0ai/mem0 • Apache-2.0（~60k stars，2026-07 复核活跃）
5	PII 识别与脱敏	Microsoft Presidio（独立服务进程）	github.com/microsoft/presidio • MIT

6	可观测性/评测平台	Langfuse v3 自托管 (OTLP 接入) 2026-08-15 复核注记: 2026-01 被 ClickHouse 收购; v3 自托管 footprint 重 (PG+ClickHouse+Redis+S3), 边缘/VPC 节点以轻量导出契约对接, 不强制随私有化镜像内置	github.com/langfuse/langfuse • MIT (ee/ 目录除外)
7	LLM 回归/红队评测	DeepEval (CI 质量门) + Promptfoo (红队与多模型对比)	github.com/confident-ai/deepeval • Apache-2.0; github.com/promptfoo/promptfoo • MIT
8	执行沙箱 (不可信代码/浏览器动作隔离)	E2B (SaaS 默认) / Daytona 自托管 (VPC, Kata 强化隔离)	github.com/e2b-dev • SDK Apache-2.0; github.com/daytonaio/daytona • AGPL-3.0 (独立进程)
9	浏览器自动化 (分层执行面, 详见 6.6)	确定性优先分层: 官方 API → 自研确定性适配器 → Stagehand 剧本 (主路径) → browser-use (长尾冷启动探索) → Skyvern (视觉抗改版, AGPL 仅独立进程) → 人工	github.com/browserbase/stagehand • MIT (2026-07 复核活跃); github.com/browser-use/browser-use • MIT (78k+ stars, 活跃); github.com/Skyvern-AI/skyvern • AGPL-3.0 (独立进程)
9b	桌面 GUI 操控 (Computer Use)	Agent-S3 (gui-agents 库) + pywinauto/pyobjc 确定性直调降级 + 虚拟输入通道	github.com/similar-ai/gui-agents • Apache-2.0 (2026-07 活跃)
9c	流程固化 (录制回放)	OpenAdapt (openadapt-flow 演示编译器)	github.com/OpenAdaptAI/OpenAdapt • MIT (2026-07 活跃)
9d	消息图谱 NL 检索 (G1)	WrenAI 独立进程 (语义层 MDL + governed text-to-SQL) 注: wren-engine 已并入主仓 core/; 旧 chat-first 应用转入 legacy/v1 分支——本方案仅用其 governed text-to-SQL 引擎能力	github.com/Canner/WrenAI • Apache-2.0 (2026-05 复核活跃)

10	计量计费	自研轻量计量（遥测 seam 逐消息）+ Lago 作独立计费进程（可选，停车场）	github.com/getlago/lago • AGPL-3.0（独立部署）
11	服务层/API	Hono + tRPC + Drizzle ORM（TypeScript 全栈同语言）	github.com/honojs/hono • MIT; github.com/trpc/trpc • MIT; github.com/drizzle-team/drizzle-orm • Apache-2.0
12	前端与端侧	React 19 + Vite + Tailwind + shadcn/ui；桌面壳 Tauri 2；小程序 Taro 4；Agent 卡片/审批组件可选复用 CopilotKit（AG-UI，仅作组件库——消息流本身由 dsh 承接）；工作台组件库 ECharts/TipTap/SheetJS/PptxGenJS/Fabric.js；报告 PDF 用 WeasyPrint（服务端）	github.com/tauri-apps/tauri • Apache-2.0/MIT; github.com/NervJS/taro • MIT; github.com/CopilotKit/CopilotKit • MIT（AG-UI 生态 2026 持续扩张）; github.com/apache/echarts • Apache-2.0; github.com/Kozea/WeasyPrint • BSD-3
13	数据存储（分层）	云端/集团：PostgreSQL 16 + pgvector + RLS；社区版/边缘节点：SQLite WAL + Litestream 备份 + sqlite-vec；对象存储：云 OSS/S3（SaaS）、SeaweedFS（VPC）	PostgreSQL License; github.com/pgvector/pgvector • PostgreSQL License; github.com/benbjohnson/litestream • Apache-2.0; github.com/asg017/sqlite-vec • MIT; github.com/seaweedfs/seaweedfs • Apache-2.0
13b	锁/幂等/缓存	首版 PG advisory lock + 幂等键表；多实例部署时引入 Redis 7（停车场）	github.com/redis/redis • RSALv2/SSPLv1/AGPLv3

14	IM 审批连接器（原生消息类型的外发通道）	自研插件（钉钉/企微/飞书/Slack 官方开放平台 SDK）	各厂商开放平台
15	调度	dsh ctx.jobs + 定时触发插件（自研薄层）	底座自带

自研边界清单（护城河仅六件 + 两薄层）：flydata-core（消息总线）、fence-engine（行动权限）、night-shift（在线时长）、review-console（原生审批消息）、model-router（计量路由）、tenancy（裁剪计费）+ 意识系统插件、行业 bundle 资产——其余全部复用。这就是「30% 插件化」的精确构成。

4 消息协议与数据模型

4.1 五元消息（业务事件）JSON Schema = 企业 Agent IM 的消息协议

PRD 的五元消息模型（Who × Context × Object × Decision × RuleImpact）转写为 dsh

SessionEventMap 扩展——业务消息与运行时事件（turn/、tool/）共存于同一 append-only 日志，

「模型可见即已记录」（G8）运行时断言自动覆盖 Agent 侧写入。文本消息 =

`decision.action="message.text"` 的退化形态；审批卡片、决策包、围栏判定、巡检异常均为本协议的原生消息类型。

```
// flydata/business_event.schema.json
{
  "$id": "cw://flydata/business-event/v1",
  "type": "object",
  "required": ["event_id", "who", "context", "object", "decision", "rule_impact", "receipt"],
  "properties": {
    "event_id": { "type": "string", "pattern": "^E-\\d+$",
      "description": "服务端单调递增，幂等键 (L1.4) " },
    "who": { "type": "object", "required": ["type", "id"],
      "properties": {
        "type": { "enum": ["human", "agent", "system"] },
        "id": { "type": "string" },
        "version": { "type": "string" } // Agent 版本，归因必需 (IM 成员身份)
      } },
    "context": { "type": "object", "required": ["tenant_id", "workspace_id", "time"],
      "description": "行业字段集由行业 bundle 扩展 (酒店: store/channel/stage; 营销: custom)",
      "properties": {
        "tenant_id": { "type": "string" }, "workspace_id": { "type": "string" },
        "time": { "type": "string", "format": "date-time" }, "channel": { "type": "string" },
        "stage": { "type": "string" } } },
    "object": { "type": "object", "required": ["type"],
      "description": "对象枚举行业化: 酒店=房型/房价/渠道/订单/客人/评价/门店/员工; 营销=账号/产品/活动/渠道/品牌/广告/内容/效果/数据/洞察/策略/工具/平台/生态/合作伙伴/渠道/品牌/广告/内容/效果/数据/洞察/策略/工具/平台/生态/合作伙伴" },
    "decision": { "type": "object", "required": ["action"],
      "properties": {
        "action": { "type": "string" },
        "before": {}, "after": {}, // 前后值, diff 视图与回滚的数据源
        "basis": { "type": "array", "items": { "type": "string" } },
        "memory_refs": { "type": "array", "items": { "type": "string" } },
        "description": "本次决策引用的组织记忆 ID 列表 (F1.4 归因闭环) " } },
    "rule_impact": { "type": "array",
      "items": { "type": "object", "required": ["rule_id", "version", "result"],
        "properties": { "rule_id": { "type": "string" }, "version": { "type": "string" },
          "result": { "enum": ["pass", "review", "blocked", "conflict"] } } },
    "receipt": { "type": "object",
      "description": "执行回执位 (F1.1/E3.7): 外部系统是否生效 + 证据快照; 无回执=未核实",
      "properties": { "synced": { "type": "boolean" }, "snapshot_uri": { "type": "string" },
        "verified_at": { "type": "string", "format": "date-time" } } },
    "model_trace": { "type": "object", "required": ["model_id"],
      "properties": { "model_id": { "type": "string" }, "tier": { "enum": ["standard", "flagship"] },
        "window": { "enum": ["peak", "off-peak"] }, "credits": { "type": "number" } },
    "links": { "type": "array", "items": { "type": "string" },
      "description": "上游消息链 (采集/记忆调用/审批) " },
    "hash": { "type": "string", "description": "前序消息哈希链字段, 防篡改 " },

```

```
}
```

4.2 主存储 DDL (PostgreSQL; 社区版/边缘同构 SQLite)

```

-- 消息库: append-only, 禁 UPDATE/DELETE (由 DB 角色权限强制)
CREATE TABLE biz_events (
    seq          BIGSERIAL PRIMARY KEY,          -- 服务端单调递增 (L1.4)
    event_id     TEXT NOT NULL,
    tenant_id    TEXT NOT NULL,
    workspace_id TEXT NOT NULL,
    session_id   TEXT NOT NULL,                -- 与 dsh 会话日志关联键 (会话=线程)
    payload      JSONB NOT NULL,                -- 4.1 的完整五元消息
    prev_hash    TEXT NOT NULL,
    hash         TEXT NOT NULL,                -- sha256(prev_hash || payload)
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (tenant_id, event_id)                -- 幂等键: 重复写入丢弃
);
CREATE INDEX idx_events_ws_time ON biz_events (workspace_id, created_at DESC);
CREATE INDEX idx_events_object ON biz_events ((payload->'object'->'type'),
                                              (payload->'object'->'id'));
CREATE INDEX idx_events_rule ON biz_events USING GIN ((payload->'rule_impact'));
-- 检索 P95≤3s (G1) 由以上索引 + JSONB GIN 覆盖结构化/自然语言双入口

-- 组织记忆 (mem0 内核之上自建的归因/回流层)
CREATE TABLE org_memory (
    memory_id    TEXT PRIMARY KEY,
    tenant_id    TEXT NOT NULL,
    scope        TEXT NOT NULL,                -- store/customer/group/shared
    kind         TEXT NOT NULL,                -- preference/pattern/sop/forbidden
    content      TEXT NOT NULL,                -- 脱敏后的模式化结论
    embedding     vector(1536),
    source_events TEXT[] NOT NULL,              -- 来源消息 (归因)
    confidence    REAL NOT NULL DEFAULT 0.5,
    status        TEXT NOT NULL DEFAULT 'active', -- active/superseded/recalled
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE memory_usage (                  -- 哪次决策用了哪条记忆 (F1.4)
    memory_id TEXT REFERENCES org_memory(memory_id),
    event_id  TEXT NOT NULL,
    used_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (memory_id, event_id)
);

-- 围栏规则 (行动权限规则包: 版本化, 可回滚; CRUD 全部留痕)
CREATE TABLE fence_rules (
    rule_id  TEXT NOT NULL,
    version  TEXT NOT NULL,

```

```

match_spec JSONB NOT NULL,      -- 对象/动作/条件 (如 房价单日变动>8%)
action     JSONB NOT NULL,      -- allow/ask/deny + 通知策略
is_baseline BOOLEAN NOT NULL DEFAULT false, -- 出厂基线: 单调守卫, 只可收紧
created_by TEXT NOT NULL, approved_event_id TEXT, -- 变更需审批留痕 (F2.4)
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
PRIMARY KEY (rule_id, version)
);

-- 一客一档 / 一店一档 (行业字段集 JSONB 承载, 行业 bundle 定义子 Schema)
CREATE TABLE profiles (
  workspace_id TEXT PRIMARY KEY,
  tenant_id    TEXT NOT NULL,
  industry     TEXT NOT NULL,      -- marketing / hotel / ...
  archive      JSONB NOT NULL,
  forbidden    JSONB NOT NULL DEFAULT '[]', -- 禁用承诺: 硬约束, 优先级最高 (L1.6)
  pii_vault    JSONB,              -- AES-256-GCM, 密钥本地半年轮换 (F1.10)
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 行业知识资产表 (高表现结构库/风格库/SOP 库统一承载位)
CREATE TABLE industry_assets (
  asset_id     TEXT PRIMARY KEY,
  tenant_id    TEXT NOT NULL, workspace_id TEXT NOT NULL,
  kind         TEXT NOT NULL,      -- script_structure / style / sop / ...
  payload      JSONB NOT NULL,
  perf         JSONB NOT NULL DEFAULT '{}',
  share_scope  TEXT NOT NULL DEFAULT 'workspace'
  CHECK (share_scope IN ('workspace', 'org', 'industry')),
  desensitized BOOLEAN NOT NULL DEFAULT false, -- 回流共享层前必须 true (L8.1)
  embedding    vector(1536),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 审批消息队列 (手势回流的数据源; 原生消息类型)
CREATE TABLE approvals (
  approval_id TEXT PRIMARY KEY,
  tenant_id   TEXT NOT NULL, workspace_id TEXT NOT NULL,
  event_id    TEXT NOT NULL,      -- 被审批的动作消息
  channel     TEXT NOT NULL,      -- inapp / dingtalk / wecom / feishu
  status      TEXT NOT NULL DEFAULT 'pending', -- pending/approved/edited/rejected/
  gesture     JSONB,              -- {decision, edited_after, reason_e
  decided_by  TEXT, decided_at   TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

```


4.3 消息写入铁律（对齐 PRD L1.1-L1.3）

① 所有消息写入必经安全网关（权限校验→PII 脱敏→高风险授权），任何模块禁止绕过直写；biz_events 表仅授予网关服务角色 INSERT 权限，从数据库层堵死旁路。② 敏感数据（订单/客资/财务）本地存储零上传；集团共享仅回流脱敏后模式（org_memory.content 写入前过 Presidio 网关，脱敏失败即拦截并记安全事件，E1.3）。③ 回滚、围栏变更、标记已知、任务终止——一切人工动作本身都是消息，禁止无痕操作（L1.1）。

5 Base Bundle 六插件详细设计（企业 Agent IM 引擎层）

Base Bundle 是自有核心资产，闭源分发。每个插件均按 dsh seam 三角（服务定义 + 提供方 + 消费者）设计，消费者通常为面向模型的工具或面向人的命令/卡片。

5.1 flydata-core（消息总线 / 数据大脑）

职责：五元消息扩展注册、档案服务、组织记忆、脱敏回流、决策回溯。**对齐：**PRD M1 全部（F1.1–F1.12）；G7（写回 ≤10s 可检索）。

子能力	实现
五元消息类型	扩展 SessionEventMap 注册 flydata/biz-event，与 dsh 运行时事件同日志共存；deriveMessages() 投影时业务消息按需在 Ask 态渲染为引用
安全网关	flydata/* 能力事件域 + 写入前校验插件；权限/脱敏/授权三段瀑布
精准检索（F1.3）	结构化过滤走 SQL 索引；自然语言入口走 WrenAI 独立进程（语义层 MDL 固化五元消息业务定义，governed text-to-SQL 只读校验）；VPC 极简场景降级为白名单模板薄自研；结果附消息编号可展开链路
组织统一记忆（F1.4）	mem0 内核（scope=workspace/agent/run）+ 自建归因层：每次记忆命中写 memory_usage；回流共享层前过 Presidio 脱敏（F1.8）
跨店/跨客继承（F1.5）	继承=复制 org_memory + 冲突检测（禁用承诺、保底价、品牌口径逐条比对）；冲突项进「需介入」，继承动作本身为消息
行为链版本化与一键回滚（F1.6）	操作链=会话消息流子序列；回滚=逆向补偿消息序列（含 OTA 回改/PMS 回写工具调用）+ ctx.sessions.fork 恢复上下文；目标版本被后续引用时强制影响面确认（E1.5，links 字段反查）
手势校准（F1.7）	审批手势写入后，校准插件消费事件更新 org_memory（批准/修改/驳回权重 1/2/3，驳回原因枚举进偏好模式）；「同一驳回模式 3 次内不再重复」由 DeepEval 回归用例锁定
PII（F1.10）	PII 入 profiles.pii_vault（AES-256-GCM，密钥本地 KMS 半年轮换）；Agent 只能经脱敏副本视图访问

留痕删除（F1.11）	删除=写入 recall 消息并将对象标记回收区（默认 30 天可配 7-90）；消息本体永不可删
-------------	---

决策包/回溯投影：清晨决策包、审计导出（F1.9，PDF/CSV）、「为什么这样改」秒级回溯（F1.12），全部是消息库的只读投影查询，不建额外报表管道（POC 门禁⑤锁定此性质）。

5.2 fence-engine（行动权限 / 规则围栏引擎）

职责：三级判定、单调基线、版本化与回滚、dry-run 校验、对象级写锁。**对齐：**PRD M2 全部（F2.1-F2.10）。判定流水线挂 dsh tools/pre-execute 瀑布，位于审批段之前。

判定器实现：规则包为 YAML DSL（行业 bundle 内 fences/*.yaml），表达式求值用 expr-lang/expr（TS 生态沙箱求值），判定器为纯函数；规则语义强业务化（报价 5%、预算 10%、保底价、批量 50 对象双人确认），不引入通用策略引擎——OPA 待单客户规则 >100 条再评估（停车场）。规则包生命周期：draft → pending_approval → active → rolled_back，每次变更附 dry-run 报告与审批事件。

关键机制	映射
单调守卫语义（F2.3）	出厂基线注册为受保护守卫，客户 patch 只能叠加更严的 deny，无法覆盖基线——直接由 dsh monotonic guard 保证，自研反而留洞
围栏版本化（F2.4）	围栏配置即 Cordis patch 文件，进 Git 版本管理；每次变更生成新版本+审批事件；回滚=checkout 旧版本 patch
dry-run 校验（F2.5）	新围栏生效前，用 fork + 消息回放对最近 10 条历史动作模拟判定，输出「若新规当时生效将如何」报告，人确认后生效
夜班围栏快照（F2.6）	夜班启动时对当前有效围栏做版本快照写入消息，决策包可回溯「当晚按哪版围栏执行」
对象级写锁（F2.7）	同一 Object（房型×日期 / 账号×内容）并发写互斥，锁表落 PG（pg_advisory_xact_lock(hashtext(object_key)))
未声明即禁写（F2.10）	preset 加载时强制校验 fence_bindings，未声明的 Agent 系统级禁止写类动作

5.3 night-shift（Agent 在线时长 / 夜班自治班组）

职责：候选清单生成、夜间自主运行、一键暂停、清晨决策包、手势回流、峰谷联动。**对齐：**PRD M4 全部（F4.1-F4.8）；G5（暂停 ≤60s）。

状态机：
[未配置] --配置完成--> [已就绪] --22:00(可配)定时--> [运行中]
[运行中] --一键暂停--> [已暂停] --恢复--> [运行中]
[运行中] --08:30(可配)--> [生成决策包] --> [已就绪]
任意态 --取消配置--> [未配置]

运行时设计	实现
18:00 候选清单 (F4.1)	分析型只读任务扫描「今日未完成+可夜间推进」，每项附预估积分（峰谷价）与命中围栏摘要；「开启夜班」注册为 ctx.commands 人类命令，不经模型轮次
夜间运行 (F4.2)	每工作区一个 headless 会话（dsh-headless profile），收件箱模型消费询盘/评论/定时器消息；每个动作过 fence-engine；不确定性只写「需介入」消息不执行（L4.2）→ 实现为 agent.inject() 注入待确认事件
一键暂停 (F4.3 / G5 ≤60s)	ctx.jobs.stop + 工具执行取消链（含浏览器动作安全中止）；暂停命令经 IM 连接器与端内双通道下发；以消息流重放保证断点续跑（kill -9 恢复=POC 门禁②）
清晨决策包 (F4.4)	对夜间消息流做三段投影——已完成（成功工具事件）/ 待审批（ask 挂起）/ 需介入（deny/异常/冲突），结构化 JSON 推送 IM； 不是报表，是日志的一次视图查询 ——即 Agent 下班留在人收件箱的「交接班消息」
自动化引擎·触发器 (F4.7)	一等公民：消息流订阅者（session/event 监听）+ 定时触发（cron job）双入口；触发规则本身是围栏管辖对象（不允许「自动化绕过审批」的暗道）；触发器 CRUD/启停全部事件化；与意识系统（6.4）构成闭环：高频任务检测→建议固化→人一键确认→生成触发器或新技能

5.4 review-console（原生审批消息 / 统一审查 + IM 审批）

职责：统一审查队列、内联 diff、多渠道审批卡片、审批手势回写。**对齐：**PRD M5 全部（F5.1–F5.7）；G6（单条 ≤2 步、整包 ≤20 条 P95 ≤5min）。

面板数据源：approvals 表 + 工具事件的 before/after 负载渲染 diff；三手势（采纳/编辑后采纳/驳回，驳回必填原因 ≤200 字枚举+自由文本）经审批 API 回写，API 鉴权由 tenancy 统一（服务端 403，前端隐藏而非置灰，L5.1）。**IM 卡片审批 (F5.4)：**自研连接器插件（钉钉/企微/飞书/Slack），卡片按钮回调 → ctx.commands 远程触发 → 审批 API；同事件同渠道幂等（UNIQUE(event_id,

channel)); 审批结果回写后卡片原地更新状态。**权限一致性 (F5.6)**: 三端共享同一鉴权中间件; 卡片回调校验 openid→成员映射, 禁止转发卡片被非授权人审批 (E5.2)。

5.5 model-router (消息生产成本 / 模型与资源调度)

职责: 分级路由、峰谷调度、降级链、逐消息计量、VPC 本地模型。**对齐**: PRD M6 全部 (F6.1-F6.8)。直接翻译 PRD 路由伪代码为 dsh ctx.llm seam 上的策略插件:

```
// model-router/src/route.ts
async function route(task: Task, ctx: Ctx): Promise {
  const hit = await ctx.flydata.memory.lookup(task); // 先查组织记忆 (F6.1)
  if (hit?.reusable) return { kind: 'reuse', memory: hit }; // 零/低消耗直接复用
  const tier = classify(task); // standard | flagship (F6.2)
  const window = inWindow(now(), '22:00-08:00') ? 'off-peak' : 'peak'; // F6.3
  const policy = await ctx.tenancy.modelPolicy(task.workspaceId); // F6.7 用户策
  const chain = policy.chainFor(tier, window); // 例 [V4-Pro, GLM-5.1, Qwen3.7-Plu
  for (const m of chain) {
    if (await healthy(m)) return { kind: 'call', model: m, window };
    emit(event('model/degraded', { from: m.id })); // L6.1 禁止静默换模型
  }
  return { kind: 'queue', until: 'off-peak' }; // 深度任务可选排队 (E6.1)
}
```

调用留痕: 每次调用 (含切换与降级) 写消息 model_trace 字段; 「关键数字必须来自工具回执, 无回执标未核实不得自动执行」(F6.8/E6.5) 实现为 post-execute 段的回执校验器。**脱敏硬约束**

(F6.6): 发往云端模型的内容必经 Presidio 网关, 在 llm 适配器出站段强制, 不可由插件绕过。**计量 (F6.5)**: 遥测 seam 逐消息记录 (模型×任务×时段×积分), 月账单=消息投影 (L6.3); 单任务消耗熔断 (E6.3) 为计量段上的预算守卫。**VPC (F6.7)**: 注册指向内网 LiteLLM 代理 (vLLM/Ollama 端点) 的适配器插件, 云端零调用。

5.6 tenancy (组织、版本与计费)

职责: 租户/工作区隔离、版本能力矩阵、积分账户、订阅与商务模式核算。**对齐**: PRD M7 全部 (F7.1-F7.7)。

数据隔离 (F7.1): 每客户/门店一个工作区 = 一个 dsh profile 工作区 + 存储提供方按 workspace_id 行级隔离 (PG RLS 强制); 渠道角色 API 层过滤, 「越权查询返回空而非 403」防侧信道 (L7.1)。**版本裁剪 (F7.2)**: 版本即 bundle 组合差异 + patch——社区版 profile 不装 Quest preset 与 night-shift bundle; 「UI 隐藏而非置灰」由前端按已加载能力渲染自然达成; 服务端越版调用强制 403 + 升级路径提示。**积分计费 (F7.3)**: 订阅+积分消耗; 账单字段=工作区/任务/模型/时段/积分/费率; 峰

谷定价由 model-router 的 window 字段天然产出。**商务模式 (F7.4)**：现金/分期（预付代扣+断供宽限 15 天+3 次提醒）/资源置换（行业化，如酒店免房置换 N=C/P 折算、核销幂等防超卖、月度对账单）；增量分成以 flydata-core 归因报告为唯一依据，口径合同化（L7.5）。**到期降级 (F7.5)**：到期降社区版能力，数据保留 90 天可导出；升级数据无缝保留——数据模型不因版本变化，天然满足。**凭据与令牌管理 (F7.7)**：dsh credentials seam 提供方替换——SaaS 用加密数据库+KMS，VPC/边缘用本地凭据存储；第三方平台令牌统一走凭据 seam，Agent 工具运行时按需注入、永不出现在提示词与事件明文里（只记引用 ID）；轮换与吊销留痕。

6 行业 Bundle 规范（行业复制 = 换成员、换群规、换皮肤）

6.1 目录结构与清单契约

一个行业 Bundle 是一个 npm 包，内含五类资产；任何第三行业按此结构填充即完成落地，不改底座：

```
workloom-{industry}/
├─ package.json          # dsh 字段声明 bundle/profile (见 6.2)
├─ cordis.patch.yml      # 本 bundle 插入的配置行（可被上层 patch 覆盖，只能更严）
├─ presets/              # 行业通讯录：角色化 Agent 预设（每 Agent 一个目录）
│   └─ pricing-agent/
│       ├─ preset.yml    # 工具集 + 审批策略 + 并发约束
│       ├─ prompt/       # 系统提示词分段（注入行业术语表/合规段）
│       └─ fence.bindings.yml # 绑定的围栏声明（未声明即禁写，F2.10）
├─ tools/                # 行业工具插件（MCP 优先，原生兜底）
├─ fences/               # 行业群规：围栏包（默认规则集 + 基线守卫标记 is_baseline）
├─ skills/               # SKILL.md 格式行业技能（官方套件）
├─ schemas/              # 行业消息语义：对象枚举、阶段枚举、档案子 Schema、事件 context
└─ ui/                   # 行业皮肤：工作台前端包（按 12.3 页面→PRD 关联总表实现）
```

6.2 package.json 契约（可直接编码）

```

{
  "name": "@workloom/hotel",
  "version": "1.0.0",
  "dsh": {
    "bundle": { "patch": "./cordis.patch.yml" },
    "profile": {
      "name": "workloom-hotel",
      "bundles": [
        "@workloom/base",          // Base Bundle 永远在最下层
        "@workloom/hotel"
      ],
      "requires": { "dsh": "0.1.x-locked" }
    }
  },
  "workloom": {
    "industry": "hotel",
    "provides": {
      "presets": ["pricing-agent", "inspection-agent", "reconcile-agent", "review-agent"],
      "tools":   ["atlas-browser", "pms-cli", "ota-channel-*"],
      "fences":  ["fences/hotel-baseline.yml"],
      "skills":  ["skills/*"],
      "schemas": ["schemas/hotel-objects.json", "schemas/hotel-stages.json"],
      "ui":      ["ui/dist"]
    }
  }
}

```

6.3 行业化五要素清单（行业 bundle 开发指南摘要）

要素	要填什么	营销版实例	酒店版实例
档案 Schema	一 X 一档字段组	一客一档七大模块	一店一档（物业/品牌规范/竞对/客群/SOP/禁用承诺）
对象与阶段 枚举	Object 类型 + 经营/ 账号阶段	账号/内容/线索/预算 × 六阶段	房型/房价/订单/评价 × 五阶段
工具集	行业动作（MCP 优先）	投放平台/社媒 API、 脚本工厂	PMS CLI、Atlas 浏览器、OTA 后台、 Computer Use

围栏包	三级规则默认集 + 基线	报价 >5% 必审、预算 >10% 必审、批量 >50 双人确认	调价 ≤8% 自动、保底价熔断、退款 ≥¥500 必审、差评必审
Agent 班组	preset 清单与分工	运营 8 + 私域 5 + 巡检/对账 2	调价/对账/巡检/评价/内容/竞对/桌面 7

6.4 技能广场与意识系统（对齐 PRD M8）

技能=SKILL.md 标准目录（跨工具通用，资产不被锁死）；官方套件做成 bundle 内 skills/；「安装后即被对应 Agent 调用」为 Skills 机制原生语义。**技能绑定围栏（F8.2）**：技能插件注册守卫，卸载即撤销——Cordis 可撤销效果的运行时保证，自研系统的高危区在这里免费获得。**零代码自定义技能（F8.3）**：自然语言三要素（触发/步骤/边界）→ 引导式生成 SKILL.md，生成物进 Git 版本管理。**意识系统（F8.4）**：自建差异化插件——消费会话消息流做高频相似任务检测（embedding 聚类+频次阈值，默认 ≥3 次/周），产出「建议固化」卡片，经人类命令一键固化。事件流上素材齐备，这是 dsh 留白的记忆/分析赛道，正是护城河位置。**评估数据飞轮（F8.7，长期预留零额外管道）**：审批手势、驳回原因、轨迹回放本就沉淀在消息流中；按 OTEL GenAI 语义周期性导出为 DeepEval/Langfuse 数据集——只声明导出契约，不做提前建设。

6.5 巡检中心（主动消息，对齐 PRD M9）

定时 jobs + 只读 preset（巡检 Agent 不加载任何写工具，「只读全自动」由工具集裁剪天然保证，L9.1）+ 异常即消息写入 + 高优事件触发 IM 推送（G3 ≤5min）+ 「一键派单」=以异常消息为输入 spawn 对应业务 Agent。「巡检失败不允许静默跳过」（L9.2）由 jobs 失败事件+重试语义保证。

6.6 执行面五级分层策略（行业工具插件的统一抽象）

L1 官方 API（Meta/TikTok/PMS 官方接口）	— 零 token，首选
L2 确定性适配器（自研胶水，复用已登录会话/CLI）	— 零 token，网站-确定性脚本
L3 Stagehand 剧本（确定性代码 + LLM 局部决策）	— 动作缓存免推理，生产主路径
L4 browser-use（LLM 全自主探索）	— 仅新平台冷启动；探明后固化为 L3 剧本
Skyvern（视觉驱动，抗页面改版）	— AGPL 独立进程，渠道适配失效时的降级路径
L5 人工接管（任务挂起 + 派单）	— 兜底，进「需介入」

统一抽象接口 PlatformAdapter（每平台 300-600 行胶水）：把各平台动作语义（发布/回复/改价/巡检项）归一为 Agent 可调用工具，注册进 ctx.tools 后被全部模式继承；桌面 GUI 场景走 Agent-S3 + pywinauto/pyobjc 直调降级，虚拟输入通道不劫持真实鼠标（L3.5）；OpenAdapt 录制回放负责人工流程→确定性剧本的转化，与意识系统构成「检测高频任务→建议固化→录制编译→剧本上线」的

完整资产化管道；执行前健康检查（页面特征探针）作为 tools/pre-execute 段实现，探针失败自动降级一层或转人工（E3.1）。

7 前端与多端（IM 客户端层）

桌面端主应用：Tauri 2 + React 19 + Vite + Tailwind + shadcn/ui（安装包小、内存低，门店低配 PC 友好），沿用 PRD 深色工作台设计，经 dsh 官方 UI 集成路径直连运行时（ctx.agents 驱动 + session/event 渲染）；POC 期先用 dsh 默认 Web UI 调试。Agent 对话/步骤卡/审批组件可选复用 CopilotKit（AG-UI 协议，仅作组件库——消息流本身已由 dsh 承接）。**小程序/移动端：**Taro 4 一套 React 代码出微信小程序（只读+审批+一键暂停）。**工作台组件库：**写作=TipTap；数据=ECharts+SheetJS；幻灯片=PptxGenJS；设计=Fabric.js；审计报告 PDF=WeasyPrint 服务端渲染（确定性渲染 $\leq 30s$ ）。**文档即事件：**工作台内的编辑、导出、设计决策均产生五元消息写入事件流（编辑器事务→防抖合并→事件），每一次交付都可回溯「谁、何时、基于什么数据、改了什么」。**自定义卡片：**审批面板、决策包三栏、Trajectory 业务化皮肤，注册为 ConversationNodeDefinition 新聊天节点类型，不 hack 前端。**IM/小程序端：**headless + api-gateway 插件暴露 REST/WS；移动审批走 review-console 连接器。

PRD 高保真稿是前端唯一视觉事实源：技术文档只定组件与数据契约，不重复描述视觉。组件命名约定 P{页码}E{角标}；页面状态规格表逐态转 Storybook 用例（默认/加载/空/错误/权限/完成后/超时）。页面↔PRD 关联总表见 12.3。

8 安全与合规设计

8.1 威胁建模（STRIDE 摘要）

威胁	场景	承接机制
伪造	伪造审批回调 / 伪造 Agent 身份	IM 回调签名校验 + openid→成员映射；Agent 身份带版本进消息 who 字段
篡改	改写历史消息 / 围栏	append-only + 哈希链（4.2）；围栏版本化 + 变更审批
抵赖	「AI 当时为什么这么决策」无法回答	「模型可见即已记录」运行时断言 + 决策链路导出（F1.9）
信息泄露	PII 进模型 / 跨租户串数据 / 脱敏失败	Presidio 出站网关强制段；PG RLS + 越权返回空；E1.3 拦截 + 批次清除
拒绝服务	任务死循环烧积分 / 磁盘写满	单任务消耗熔断（E6.3）；磁盘 80% 告警、写满转只读保护 + 执行全转必审（E1.1）
提权	社区版调 Quest / 客户放宽基线围栏 / PTC 子调用绕审批	服务端版本校验 403；单调守卫 deny 不可逆；PTC 子调用与普通调用同一条流水线（POC 门禁①实测无后门）

8.2 插件供应链安全（P0）

生产部署仅允许签名白名单插件；第三方插件与 MCP Server 入库前代码审计；行业 bundle 只允许白名单内 MCP Server 且每个 Server 的工具按围栏级别逐个授权；创造模式（自指运行时修改）在企业部署中默认禁用，仅对内部高级运维经显式开关开放。模型网关侧强化（2026-08-15 复核新增）：LiteLLM 按镜像哈希 pin（非浮动 tag）+ cosign 验签 + 仅内网监听 + 虚拟密钥分域轮换——直接回应 2026-03 PyPI 投毒事件与 CVE-2026-42208。

8.3 合规清单

操控留痕 100%（运行时不变量 G8）；审计导出 PDF/CSV 留痕（F1.9）；PII 加密+密钥本地半年轮换（F1.10）；VPC 数据不出企业（控制面/数据面分离，S-F2）；观测数据以 OTEL GenAI 语义导出 Langfuse，避免被私有格式锁定；三端权限模型一致（F5.6）。

9 非功能指标与 SLO 映射（每条 SLO 指认到机制）

口径	目标	承接机制	降级路径
G1 消息检索	P95 ≤3s	PG 索引+JSONB GIN；NL 入口 WrenAI 语义层	超时降级结构化过滤表 单（E1.6）
G10 驾驶舱 首屏	单店 ≤3s；聚合 ≤50 工 作区 ≤5s	三投影并行预取 + 分块渲染	骨架屏分块加载，不白 屏
G2 夜班询盘 首响	≤30s	headless 收件箱即时唤醒 + 围栏 直通	超时转「需介入」+ 通 知
G3 巡检异常 推送	≤5min	jobs 定时 + 高优事件 IM 推送	推送失败重试队列
G4 高优消息 通知	≤30s	行业包高优条件校验过即触发通 知命令	通道降级（IM→短信）
G5 一键暂停	端到端 ≤60s（含浏览器 安全中止）	jobs stop + 工具取消链	超时升级 P0 告警并强 制隔离会话
G6 清晨决策 包审批	单条 ≤2 步；整包 ≤20 条 P95 ≤5min	卡片内联审批 + 批量采纳	端内面板兜底
G7 消息写回 可检索	≤10s	同步写库 + 索引即时生效	写满转只读保护 （E1.1）
G8 留痕覆盖 率	100%	运行时断言「模型可见即已记 录」	无降级——不达标构建 不得发布
G9 夜班成本	夜间旗舰 ≤ 标准费率 20%	model-router 峰谷窗口	排队等待而非升档 （E4.4）
G11 会话并 发	单工作区 ≤10 并发 （POC 门禁⑦）	Spawn/Fork + 调度插件并发上 限 + 对象写锁	超出排队并可见（L3.1）

10 风险与对策

风险	等级	对策
dsh v0.1 破坏性变更	高	锁定版本 + fork 内部镜像；定制全走 patch/自有 bundle；核心接口封装防腐层；「上游跟进」责任人按季度评估升级，升级前跑契约测试套件
上游战略摇摆（回收为自家客户端）	中	MIT 保证 fork 底线；业务资产 100% 在自有插件与数据层；最坏情况迁移成本=重写运行时粘合层
深插件链性能/调试复杂度	中	POC 期建压测基线与 profiling 流程；生产精简插件集（为行业 preset 瘦身）
插件/MCP 供应链安全	中	签名白名单 + 入库审计 + 禁用创造模式（8.2，P0）
LiteLLM 供应链与漏洞史（2026-03 PyPI 投毒、CVE-2026-42208）	中	镜像哈希 pin（非浮动 tag）+ cosign 验签 + 仅内网监听 + 虚拟密钥分域轮换 + 季度升级评审；仅作 VPC 模型代理，不暴露公网
Langfuse 被 ClickHouse 收购后的路线不确定性	低	观测数据以 OTEL GenAI 标准语义导出，替换观测后端不改业务代码；VPC/边缘默认不内置其重足迹自托管栈
文档/接口不成熟导致探索成本	中	以官方架构文档 + --dump-config 为准绳；关键行为写契约测试锁定
竞品跟进（其他开源 Harness）	低	护城河在行业 bundle 与消息总线，不在运行时选型；保持 seam 隔离可二次迁移

11 停车场（按需引入，写明触发条件）

组件	触发条件
Kafka + ClickHouse 消息管道	单租户消息写入持续 >10 万条/日，PG 索引维护成本超过收益
Temporal 持久化工作流	出现跨日、多系统补偿的 saga 类业务流程（如免房置换全链路自动化）
K8s + 服务网格	VPC 客户要求多节点高可用且自带运维团队
RBAC 细粒度权限引擎（OPA）	企业客户提出角色 × 对象 × 动作三维权限矩阵需求（当前两级角色足够）
Lago 计费进程	需要标准发票、支付编排、多币种时引入（独立进程）
第二模型复核段	高风险动作跨厂商模型独立复核（审批瀑布加一段即可，多供应商适配层已就位）
OPA 通用策略引擎	单客户围栏规则 >100 条、薄自研判定器维护成本超过引入成本时
Keycloak / SSO	企业版客户提出 SSO/SCIM 需求时（首版 JWT+RLS 薄自研足够）
Redis	多实例部署、PG advisory lock 成为瓶颈时
n8n / Windmill	触发逻辑已长在编排层与 ctx.jobs，引入即过度设计；仅当出现大量非技术人员可视化编排诉求时再评估
A2A / Agentic Mesh 对接	行业系统（PMS/投放平台）以 Agent 形态出现时，消息总线成为身份与记忆层——本网络从「人机混编 IM」自然演进为「多方 Agent 互联网络」
行业 bundle 灰度发布通道	行业 bundle 数 >3 且客户实例 >50 时：bundle 版本通道（stable/beta）+ 按租户灰度 + 一键回滚（底层机制已就位：bundle 即版本化 npm 包 + patch 可回滚，仅缺发布编排界面）

12 面向 AI Coding 的执行设计

12.1 模块依赖拓扑（非排期，仅表达依赖先后）



12.2 POC Decision Gate（七条全绿才进入正式建设）

① 围栏瀑布在 PTC 子调用下仍生效（无后门实测）；② kill -9 后任务断点续跑且幂等；③ 任取 100 个动作，留痕与决策链路完整率 100%；④ 一键暂停端到端 ≤60s；⑤ 清晨决策包可由纯日志投影生成，无额外报表管道；⑥ 行业 preset 与 dsh 升级解耦验证（模拟上游 breaking change，仅 patch 层受影响）；⑦ 单工作区 10 并发 Quest 压测达到 G 系列口径。任一红 → 回退重新评估路径 B/C。

12.3 页面 ↔ PRD 关联总表（锚定 PRD V2.1 P 章节）

前端页面	PRD 锚点	角标/结构	数据契约来源
IM 主界面（派遣中心首页）	P1	P1E1-P1E7	flydata-core 三投影 + tenancy 版本裁剪
任务线程会话（Quest 工作台）	P2	P2E1-P2E6	dsh session/event 渲染 + 审批 API

清晨决策包消息（移动端）	P3	P3E1-P3E5 三栏结构	night-shift 决策包投影 JSON
待办消息中心（统一审查面板）	P4	P4E1-P4E6 diff/采纳/驳回	review-console + approvals 表
群权限与围栏配置	P5	P5E1-P5E4 规则 CRUD/版本	fence-engine 版本化 API
能力商店（技能广场）	P6	P6E1-P6E4 套件/安装/绑定	bundle skills 注册表
行业装配合	P7	P7E1-P7E4 六槽位/校验	bundle 清单契约（6.2）
行业工作台页	行业 PRD P 章节	各页状态规格表	行业 bundle ui/ 包
渠道总控台	Y-P9	多客户工作区聚合	tenancy 隔离 + 聚合投影

约定：组件命名 `P{页码}E{角标}`；每页状态规格表逐态转 Storybook 用例；视觉以 PRD 高保真稿为唯一事实源，本方案不重复描述视觉。

12.4 任务卡规范（AI Coding 派发单元）

```
task_card:
  id: CW-BASE-017
  refs: [F1.6, L1.1, E1.5] # 对齐的 PRD V2.1 编号
  module: flydata-core
  inputs: { schema: "flydata/business-event/v1", tables: [biz_events] }
  outputs: { api: "POST /v1/rollback", events: ["flydata/rollback"] }
  acceptance: # 可执行断言，不接受「看上去对」
    - "抽样 100 消息五元字段完整率 == 100%"
    - "回滚动作产生新消息且原消息未被修改（append-only 校验）"
    - "目标版本被后续引用时返回影响面清单且需确认（E1.5）"
  forbidden: # 禁止事项
    - "禁止绕过安全网关直写 biz_events"
    - "禁止无痕回滚 / 禁止 UPDATE biz_events"
  tests: { unit: "...", integration: "...", redteam: "promptfoo 越权工具调用集" }
```

12.5 测试策略分层

单元：围栏判定纯函数、NL2Where 模板、脱敏规则（中文 recognizer 覆盖率）；**集成：**消息写入→投影→审批→回流闭环；kill 恢复幂等；PTC 子调用过瀑布；**契约：**dsh 接口防腐层契约测试（上游升级前先跑）；消息 schema 版本兼容；**红队：**Promptfoo（提示注入/PII 泄漏/越权工具调用/围栏绕过尝试）；**演练：**POC 七条门禁转为常驻回归；「真机连跑 N 晚」夜班全链路演练通过后才允许开启自动执行。

13 技术新增量（PRD 未写、技术必需，单列呈报）

#	新增项	理由
A1	消息哈希链（prev_hash/hash）	PRD 要求「禁止无痕回滚/留痕 100%」未定防篡改机制；哈希链是最轻实现
A2	WrenAI NL 检索独立进程接入 + VPC 降级白名单模板	PRD 要求自然语言检索（F1.3）未定实现；语义层治理比裸 NL2SQL 更稳
A3	LiteLLM 作为 VPC 本地模型代理（哈希 pin + cosign 验签）	PRD 要求 VPC 本地模型（F6.7）未定承载组件；2026 供应链事件后强化 pin 纪律
A4	OTEL GenAI → Langfuse 导出映射层	避免观测数据锁定私有格式，客户可用标准工具回归评测
A5	插件签名与白名单基础设施	8.2 的落地件，PRD 只写了权限合规目标
A6	消息 schema 版本规约（v1 冻结 + 迁移器约定）	多行业 bundle 长期演化的前提
A7	审批卡片回调签名校验与 openid 映射	IM 审批的安全前提，PRD 只写功能
A8	执行面五级分层与 PlatformAdapter 抽象	PRD 写了跨应用执行能力（F3.9）未定分层降级策略
A9	边缘节点存储栈（SQLite WAL + Litestream + sqlite-vec）	PRD 本地优先（A6/S-F2）的落地件

14 需求 → 实现对齐总表（自查全覆盖 · 挂 PRD V2.1 编号）

PRD V2.1 模块	编号范围	承接组件	本方案章节
M1 消息总线 (flydata-core)	F1.1–F1.12, L1.x, E1.x, US1.x	flydata-core + 安全网关 + mem0 归因层	4 / 5.1
M2 行动权限 (fence- engine)	F2.1–F2.10, L2.x, E2.x	fence-engine + dsh monotonic guard	5.2
M3 会话三态与任务派 遣	F3.1–F3.9, L3.x, E3.x	dsh preset/Spawn/Fork + 行业 工具插件 + 对象写锁	2.3 / 6.6
M4 Agent 在线时长 (night-shift)	F4.1–F4.8, L4.x, E4.x	night-shift + automation-engine	5.3
M5 原生审批消息 (review-console)	F5.1–F5.7, L5.x, E5.x	review-console + IM 连接器 + approvals 表	5.4 / 7
M6 模型与资源调度	F6.1–F6.8, L6.x, E6.x	model-router + LiteLLM (VPC)	5.5
M7 组织、版本与计费	F7.1–F7.7, L7.x, E7.x	tenancy (+Lago 停车场)	5.6 / 11
M8 能力通讯录 (技能 +意识)	F8.1–F8.7, L8.x, E8.x	Skills 机制 + 意识插件 + 触发器	6.4
M9 主动消息 (巡检中 心)	F9.1–F9.4, L9.x, E9.x	巡检 jobs + 只读 preset	6.5
安全与私有化专章	S-F1–S-F8, S-L1, S-E1, S-G1	控制面/数据面分离 + seam 提供 方 + 边缘存储栈	2.2 / 8 / 13-A9
P1–P7 页面交互稿	P1E*/P2E*/…/P7E*	前端 + 页面↔PRD 关联总表	7 / 12.3
全局 G 系列口径	G1–G11	第 9 章 SLO 映射	9

自查结论：PRD V2.1 的 P0/P1 能力均有组件承接；P2 项（技能使用看板 F8.5、评估数据飞轮 F8.7、开放生态）按 PRD 优先级落入后期 bundle 迭代，架构已留位（分支=fork 语义、生态=dsh-plugin 分

发、导出契约已声明)。行业 PRD (H-/Y- 编号) 对齐关系与 V2.0 一致, 未变。

15 结语

WorkLoom 的全部工程投入收敛到三处护城河：**消息总线**（flydata-core）、**行动权限**（fence-engine）、**行业 bundle 的 know-how 装配机制**；Agent 运行时这个 2026 年起不再构成差异却极其昂贵的部分，交给正在快速成为公共地基的 dsh。

PRD V2.1 把「消息、行动权限、审批手势」定义为第一性资产——这恰好是一张企业 Agent IM 最核心三件套的业务化表达：消息协议（五元事件）、权限系统（围栏即「谁能动手」）、原生消息类型（审批卡片即一等公民）。本方案做的，是让这张「人与 Agent 同群、同会话、同一套消息协议」的协作网络，站在有形式化保证的运行时之上，成为可复制到 N 个行业的标准件。传统 IM 把沟通数字化，WorkLoom 把行动数字化——这套技术方案就是「把行动数字化」的工程实现。